

Monthly Coverage of Crime Articles in 2001

Section 1: How many New York Times articles were written about criminal activity between 2000 and 2010, and how do article trends correlate with significant FBI policy changes or high-profile criminal cases during that time?

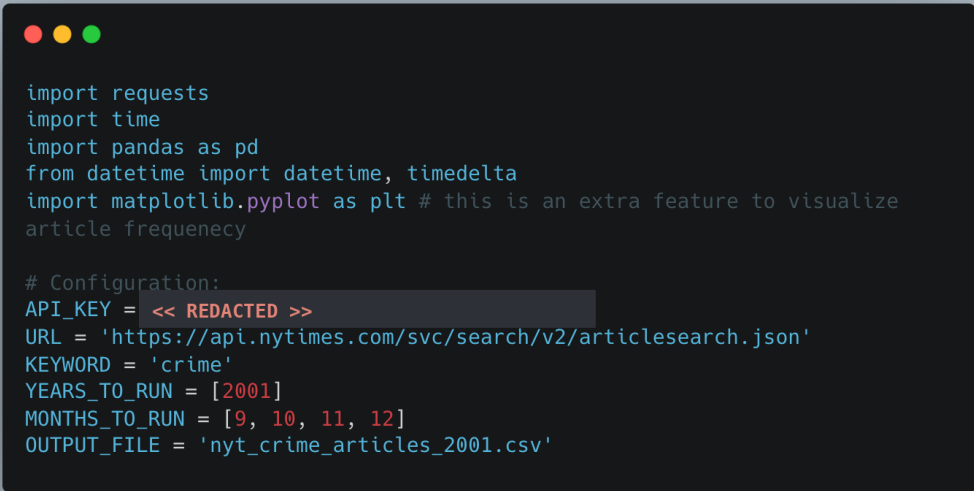
This project investigates how crime-related coverage in the New York Times fluctuated throughout 2001. Specifically, it explores whether article volume reflects reactions to significant national security events such as the 9/11 attacks and the Patriot Act.

Using the NY Times Article Search API, my process is as follows,

- I extracted and analyzed every article that mentioned “crime” between January 1, 2001 and December 31, 2001 limiting results to the first 3 pages of each month to stay within rate limits.
- I transformed the data into a structured CSV file and visualized daily publication counts with important event annotations.

Section 2: Code Process

Import Libraries and Configuration

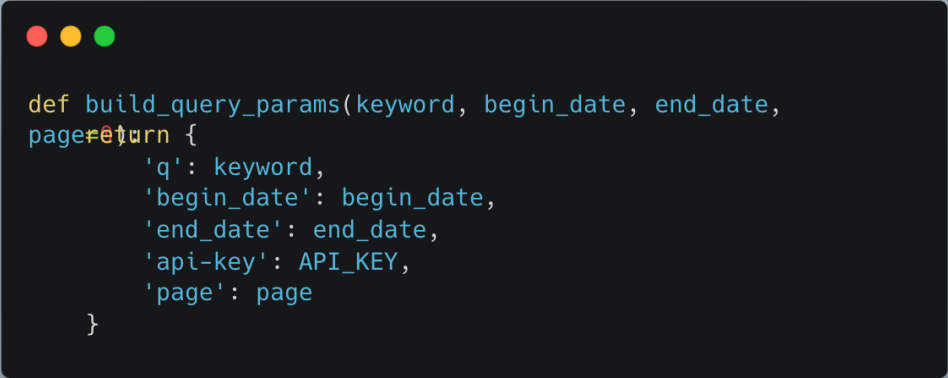


```
import requests
import time
import pandas as pd
from datetime import datetime, timedelta
import matplotlib.pyplot as plt # this is an extra feature to visualize
article frequency

# Configuration:
API_KEY = << REDACTED >>
URL = 'https://api.nytimes.com/svc/search/v2/articlesearch.json'
KEYWORD = 'crime'
YEARS_TO_RUN = [2001]
MONTHS_TO_RUN = [9, 10, 11, 12]
OUTPUT_FILE = 'nyt_crime_articles_2001.csv'
```

In this section, I import Python libraries for web requests (requests), data manipulation (pandas, datetime), and visualization (matplotlib). I also set up global variables including my NYT API key, query keyword (“crime”), and time range (2001 only, and date range from September-December).

Build API URL Using Parameters



```
def build_query_params(keyword, begin_date, end_date,
page):
    return {
        'q': keyword,
        'begin_date': begin_date,
        'end_date': end_date,
        'api-key': API_KEY,
        'page': page
    }
```

Here, I define a function that builds a dictionary of parameters for querying the NYT API. This includes the keyword, start/end dates, page number, and API key.

Extraction Setup

```
def extract_articles(year):
    """
    Extracts crime-related articles from the NYT API for the given year
    and selected months.
    Returns a list of article dictionaries.
    """
    articles = []
    for month in MONTHS_TO_RUN:
        # format start and end dates for the month
        begin_date = f"{year}-{month:02d}01"
        end_date = (datetime(year, month, 1) +
                    timedelta(days=32)).replace(day=1) - timedelta(days=1)
        end_date = end_date.strftime("%Y%m%d")

        for page in range(0, 3): # limit to first 3 pages per month
            print(f"Fetching {year}-{month:02d} page {page}...")
            params = build_query_params(KEYWORD, begin_date, end_date,
            page)

            try:
                response = requests.get(URL, params=params)
                if response.status_code != 200:
                    print(f"[{year}-{month:02d} page {page}] Error
{response.status_code}: {response.text}")
                    break

                json_data = response.json()
                docs = json_data.get("response", {}).get("docs")
                if not docs:
                    print(f"No docs returned for {begin_date} to
{end_date}")
                    break

                # extract and store relevant data from each article
                for doc in docs:
                    articles.append({
                        "pub_date": doc.get("pub_date", "")[:10],
                        "headline": doc.get("headline", {}).get("main",
                        ""),

                        "section": doc.get("section_name", ""),
                        "url": doc.get("web_url", "")
                    })

            except Exception as e:
                print(f"Exception on {begin_date} page {page}: {e}")
                continue

            time.sleep(12) # safer rate limit pause
    return articles
```

This function loops through the target year (2001) and calls `extract_articles()` for each month. It compiles all results into a master list.

Extract Articles for Each Month

```
def extract_all_articles():
    all_articles = []
    for year in YEARS_TO_RUN:
        print(f"Extracting for {year}...")
        yearly_articles =
extract_articles(year)extend(yearly_articles)
    return all_articles
```

This is the heart of the ETL process. It loops through each month of 2001 and sends up to 3 paged API requests per month. After each call, it analyzes the article metadata and appends cleaned entries to a list. A 12-second delay is included to comply with NYT's free-tier rate limit.

Transform the Data

```
def transform_articles(raw_articles):
    df = pd.DataFrame(raw_articles)
    if df.empty:
        print("No valid data to transform.")
        return df
    df.drop_duplicates(subset=["url"], inplace=True)
    df["pub_date"] = pd.to_datetime(df["pub_date"],
error=UnicodeError)
```

This function converts the raw article list into a Pandas DataFrame, removes duplicate URLs, and parses publication dates into a usable datetime format.

Save Data to CSV

```
def load_to_csv(df, output_path):  
    if df.empty:  
        print("Nothing to save – DataFrame is empty.")  
        return  
    df.to_csv(output_path, index=False)  
    print(f"Saved {len(df)} articles to  
{output_path}")
```

Once transformed, the data is saved to `nyt_crime_articles_2001.csv`, allowing for visualization in Excel or Python.

Annotate Significant Events (Part of Visualizing Article Frequency)

```
def annotate_events(ax):  
    events = {  
        '2001-09-11': '9/11 Attacks',  
        '2001-10-26': 'Patriot Act Signed',  
    }  
  
    for date_str, label in events.items():  
        date = pd.to_datetime(date_str)  
        ax.axvline(date, color='red', linestyle='--', alpha=0.7)  
        ax.text(date, ax.get_ylim()[1] * 0.95, label, rotation=90,  
                fontsize=8, color='red', ha='right')
```

To correlate article spikes with real-world events, I annotated key dates on the plot:

- September 11, 2001 – Terrorist attacks on the World Trade Center and Pentagon •
- October 26, 2001 – Signing of the USA PATRIOT Act

These vertical lines are labeled in red for visibility.

Sugar Feature: Visualizing Article Frequency

```
def plot_article_frequency(df):  
    """  
    Generates a line plot showing the number of articles published per  
    day.  
    """  
    if df.empty:  
        print("No data to plot.")  
        return  
  
    # Count articles per day  
    count_by_date = df['pub_date'].value_counts().sort_index()  
  
    plt.figure(figsize=(14, 6))  
    ax = plt.gca()  
    ax.plot(count_by_date.index, count_by_date.values, marker='|',  
linestyle='-', label='NYT Crime Articles')  
  
    annotate_events(ax)  
  
    plt.title(f"NYT Articles on '{KEYWORD}' (2001) with Major Event  
Overlays")  
    plt.suptitle("Mahima Ullah – Final Project", fontsize=10, x=0.99,  
ha='right')  
    plt.xlabel("Date")  
    plt.ylabel("Number of Articles")  
    plt.legend()  
    plt.grid(True)  
    plt.tight_layout()  
    plt.xticks(rotation=45)  
  
    # Optional: Save the plot as an image  
    plt.savefig("nyt_crime_trends_2001.png")  
    plt.show()
```

This bonus feature plots the number of articles published per day throughout 2001. It uses a simple line graph to highlight trends and annotate major events. I also include a title and subtitle with my name and save the plot as a PNG.

Run the Full ETL Pipeline

```
def run_etl():
    """
    Runs the entire ETL process:
    - Extract articles
    - Transform into DataFrame
    - Load to CSV
    - Plot article frequency
    """
    print("Starting ETL process...")
    raw_articles = extract_all_articles()

    if not raw_articles:
        print("No articles were extracted – likely due to API
        issues.")
        return

    transformed_df = transform_articles(raw_articles)
    load_to_csv(transformed_df, OUTPUT_FILE)
    plot_article_frequency(transformed_df) # show graph

if __name__ == "__main__":
    run_etl()
```

The `run_etl()` function runs the entire ETL pipeline: it extracts all articles, transforms the data, saves it to a CSV file, and finally plots the daily article frequency.

Section 3: Findings The chart revealed noticeable spikes in coverage around September 2001, directly aligning with the 9/11 attacks. Another subtle rise shows in late October, after the passage of the Patriot Act, reflecting national concerns over surveillance, terrorism, and crime.

The data supports my hypothesis that high-profile events and FBI policy shifts influence article volume. While the keyword “crime” can appear in many contexts, this simple frequency analysis shows that there is a much more meaningful correlation between media output and real-world events.

Section 4: Next Steps *If I continue this project, I would:*

- Expand the date range to include 2000–2002 for pre/post-9/11 comparison
- Apply keyword combinations like “crime” + “terrorism” for greater specificity
- Visualize trends in a dashboard or interactive timeline